

NAME :G.AKSHAYA

COURSE CODE : CSA0619

REG NO: 192521156

COURSE NAME : DESIGN

AND ANALYSIS OF

ALGORITHMS

CO4 - ASSESSMENT TOOL – 01

Q2. Unbounded Knapsack

Aim:

To apply the **Dynamic Programming** technique to solve the Unbounded Knapsack problem, where each type of item can be selected multiple times without restriction. The objective is to maximize the total profit while ensuring that the total weight does not exceed the given knapsack capacity.

Problem Statement:

A manufacturing unit produces different types of items. Each item has a specific weight and profit. An unlimited number of each type of item is available, meaning the same item can be selected multiple times.

Given the weights, values, and maximum capacity of the knapsack, the objective is to determine the combination of items that produces the maximum possible profit without exceeding the available capacity.

The problem differs from the 0/1 Knapsack problem because in 0/1 Knapsack each item can be selected at most once, whereas in Unbounded Knapsack, the same item can be selected any number of times.

Dynamic Programming is used because the problem contains overlapping subproblems and optimal substructure.

Objective:

- Maximize the total profit.
- Allow unlimited selection of every item.
- Ensure the total weight does not exceed the given capacity.
- Apply Dynamic Programming to solve the problem efficiently.
- Avoid unnecessary repeated calculations.
- Determine the optimal combination of items.

- Analyze the time and space requirements of the algorithm.
- Verify the maximum profit using the given sample input.

Dynamic Programming Approach :

Let:

$dp[w]$ = Maximum profit that can be obtained with capacity w

Initially, all values of the DP array are set to zero because a capacity of zero gives zero profit.

For every capacity from 1 to W , each item is checked.

If the weight of an item is less than or equal to the current capacity, the following recurrence is used:

$$dp[w] = \max(dp[w], \text{value}[i] + dp[w - \text{weight}[i]])$$

The important point is that the item can be selected again. Therefore, the DP array uses the already calculated value $dp[w - \text{weight}[i]]$.

Why Unbounded Knapsack?

In this problem, items are reusable.

For example, Item 2 has:

Value = 30

Weight = 10

Since the capacity is 100, Item 2 can be selected:

$$100 / 10 = 10 \text{ times}$$

Therefore:

$$\text{Total Weight} = 10 \times 10 = 100$$

$$\text{Total Profit} = 10 \times 30$$

$$= 300$$

The complete capacity is utilized and the resulting profit is 300.

Working of the Algorithm :

Step 1: Initialize DP Array

For capacity values from 0 to 100, initialize the DP array with zero.

$$dp[0] = 0$$

$$dp[1] = 0$$

$$dp[2] = 0$$

...

$$dp[100] = 0$$

Step 2: Check Item 1

Item 1:

$$\text{Weight} = 5$$

$$\text{Value} = 10$$

For capacity 5:

$$dp[5] = 10$$

For capacity 10:

$$dp[10] = 20$$

Because Item 1 can be used repeatedly.

For capacity 100:

$$100 / 5 = 20 \text{ items}$$

$$\text{Profit} = 20 \times 10$$

$$= 200$$

Step 3: Check Item 2

Item 2:

$$\text{Weight} = 10$$

$$\text{Value} = 30$$

For capacity 10:

$$\begin{aligned} dp[10] &= \max(20, 30) \\ &= 30 \end{aligned}$$

For capacity 20:

$$dp[20] = 60$$

Similarly, at capacity 100:

$$dp[100] = 300$$

Step 4: Check Item 3

Item 3:

$$\text{Weight} = 15$$

$$\text{Value} = 20$$

Although Item 3 can also be selected multiple times, its profit-to-weight ratio is lower than Item 2.

Therefore, it does not improve the maximum profit obtained using Item 2.

Profit-to-Weight Analysis:

Item	Value	Weight	Value/Weight
Item 1	10	5	2
Item 2	30	10	3
Item 3	20	15	1.33

Item 2 provides the highest profit per unit of weight.

Therefore, selecting Item 2 repeatedly provides:

$$10 \times \text{Item 2}$$

Total:

$$\text{Weight} = 100$$

Profit = 300

Optimal Solution :

The optimal combination is:

Item 1 = 0 times

Item 2 = 10 times

Item 3 = 0 times

Therefore:

$$\begin{aligned}\text{Total Weight} &= (0 \times 5) + (10 \times 10) + (0 \times 15) \\ &= 100\end{aligned}$$

And:

$$\begin{aligned}\text{Total Profit} &= (0 \times 10) + (10 \times 30) + (0 \times 20) \\ &= 300\end{aligned}$$

Hence:

Maximum Profit = 300

Algorithm:

Step 1: Start.

Step 2: Read the number of items n .

Step 3: Read the values/profits of all items.

Step 4: Read the weights of all items.

Step 5: Read the maximum capacity W .

Step 6: Create a DP array of size $W + 1$.

Step 7: Initialize all DP values to 0 .

Step 8: For each capacity from 1 to W , check every item.

Step 9: If the weight of the current item is less than or equal to the current capacity, calculate:

$\text{value}[i] + \text{dp}[\text{capacity} - \text{weight}[i]]$

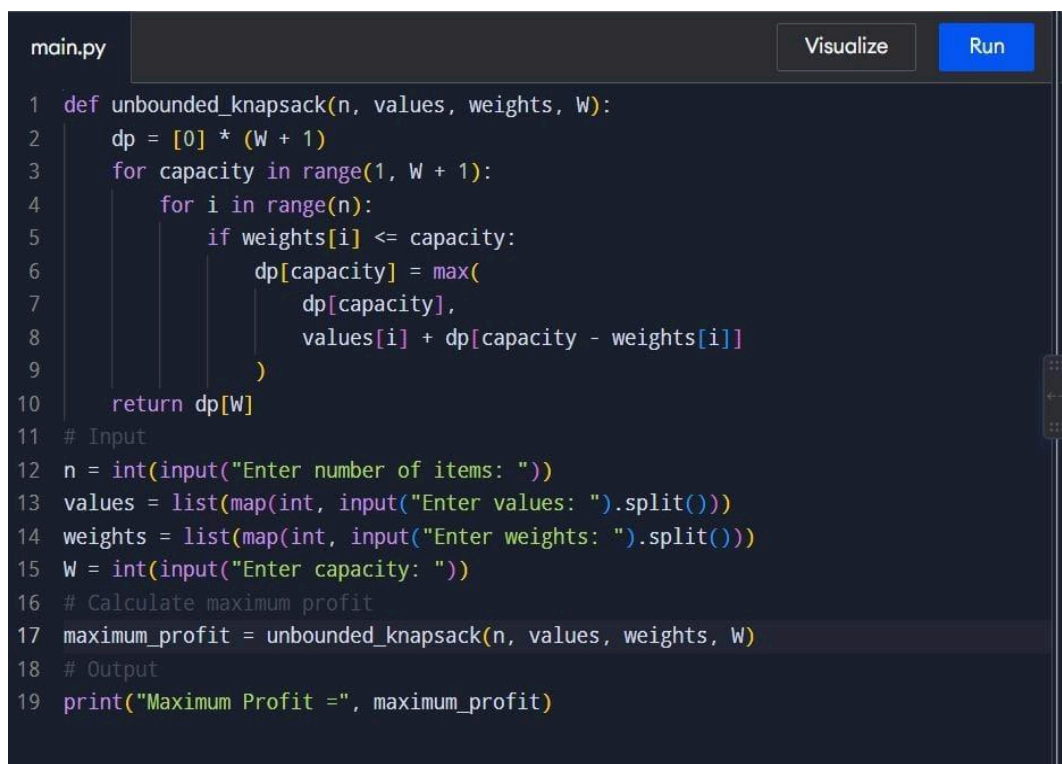
Step 10: Store the maximum value in the DP array.

Step 11: Continue until capacity W is processed.

Step 12: Display $\text{dp}[W]$ as the maximum profit.

Step 13: Stop.

SOURCE CODE:

A screenshot of a code editor window titled 'main.py'. The editor has a dark background with light-colored text. On the right side of the editor, there are two buttons: 'Visualize' and 'Run'. The code is written in Python and implements an unbounded knapsack algorithm. It defines a function 'unbounded_knapsack' that takes parameters 'n', 'values', 'weights', and 'W'. The function initializes a DP array 'dp' of size 'W+1' and iterates over capacities from 1 to 'W'. For each capacity, it iterates over items from 0 to 'n-1'. If an item's weight is less than or equal to the current capacity, it calculates the maximum value between the current value at that capacity and the value plus the item's value. Finally, it returns 'dp[W]'. Below the function, there is input handling code that prompts the user for the number of items, values, weights, and capacity, and then prints the maximum profit.

```
main.py Visualize Run
1 def unbounded_knapsack(n, values, weights, W):
2     dp = [0] * (W + 1)
3     for capacity in range(1, W + 1):
4         for i in range(n):
5             if weights[i] <= capacity:
6                 dp[capacity] = max(
7                     dp[capacity],
8                     values[i] + dp[capacity - weights[i]]
9                 )
10    return dp[W]
11 # Input
12 n = int(input("Enter number of items: "))
13 values = list(map(int, input("Enter values: ").split()))
14 weights = list(map(int, input("Enter weights: ").split()))
15 W = int(input("Enter capacity: "))
16 # Calculate maximum profit
17 maximum_profit = unbounded_knapsack(n, values, weights, W)
18 # Output
19 print("Maximum Profit =", maximum_profit)
```

OUTPUT:

```
Output

Enter number of items: 3
Enter values: 10 30 20
Enter weights: 5 10 15
Enter capacity: 100
Maximum Profit = 300

=== Code Execution Successful ===
```

Complexity Analysis:

Case	Complexity
Time Complexity	$O(n \times W)$
Space Complexity	$O(W)$

Time Complexity:

The outer loop runs for every capacity from 1 to W .

The inner loop checks all n items.

Therefore:

Time Complexity = $O(n \times W)$

For the given problem:

$n = 3$

$W = 100$

So the number of main DP iterations is proportional to:

$3 \times 100 = 300$

Space Complexity:

Space Complexity:

Only one DP array of size $W + 1$ is required.

Therefore:

Space Complexity = $O(W)$

Result:

The Unbounded Knapsack problem was successfully solved using the Dynamic Programming technique. Since each item can be selected multiple times, the algorithm repeatedly considers the available items for every possible capacity. For the given input, Item 2 with value 30 and weight 10 is selected 10 times, giving a total weight of 100 and a maximum profit of 300.

Thus, the final result is:

Maximum Profit = 300